

Module 8: Classes and Objects

Introduction to C++ Programming — A Beginner's Course

Module Overview

This module reframes everything students have learned so far — data (Modules 2, 5) and behavior (Modules 6–7) — as two halves of one idea: a class. The pedagogical arc should move from the familiar (struct, which they already know) to the new (private access, methods, constructors), rather than introducing OOP vocabulary cold. Encapsulation is the one idea worth over-explaining, since 'why hide data at all?' is a genuine, reasonable question beginners ask.

Suggested Duration: 90 minutes

Learning Objectives

By the end of this module, students will be able to:

- Explain a class as data (member variables) plus behavior (methods) bundled together.
- Distinguish a class (blueprint) from an object/instance (an actual variable of that type).
- Use private and public access specifiers, and explain the purpose of encapsulation.
- Write getter and setter methods.
- Write and use constructors, including recognizing when a default constructor must be added explicitly.
- Use `this->` correctly when a parameter name shadows a member variable.

Prerequisites

- Module 5 (struct) and Modules 6–7 (functions) — this module explicitly builds on both.

Materials & Instructor Prep

- 08-classes-and-objects/examples/02_encapsulation.cpp preloaded, with the commented-out illegal access line ready to uncomment live.

Key Vocabulary

Term	Definition
Class	A blueprint bundling data (member variables) and behavior (methods) into one type.
Object / instance	An actual variable created from a class blueprint.
Encapsulation	Hiding a class's internal data and exposing only controlled access through its methods.
Access specifier	public or private — controls whether code outside the class can access a member.
Getter / setter	A public method that reads (getter) or writes (setter), usually with validation, a private member.
Constructor	A special method, named after the class, that runs automatically when an object is created.
this	Inside a method, a reference to the specific object the method was called on.

Lesson Outline & Instructor Talking Points

1. From struct to class — start from what they know (15 min)

- Open with the Module 5 Student struct on the board. Ask: 'What if we wanted this to also know HOW to compute its own average, not just store the raw scores?' — that question is the entire motivation for the module.
- Live-build class BankAccount with public members and methods (deposit, withdraw, printStatement) as a direct, minimal-surprise upgrade from struct.

2. Why hide anything? Motivating encapsulation honestly (20 min)

- Ask directly: 'Right now, any code anywhere could write `account.balance = -500`;. Is that a problem? Why?' Let students articulate the problem themselves before naming the solution.
- Convert balance to private, add deposit/getBalance methods, and show the now-illegal direct access producing a compiler error live (uncomment the line in the example).
- Land the principle explicitly: 'private lets you GUARANTEE a rule — like balance never goes negative — because there's no other way to touch it except through code you wrote and trust.'

Instructor note: *This is the module's most important conceptual moment. Students who only memorize 'private members need getters' without understanding WHY will misapply the pattern (e.g. writing a*

getter/setter for every single field with no added validation, defeating the purpose).

3. Constructors (20 min)

- Motivate directly: 'Setting every field manually after creating the object, like we did with structs, is error-prone — easy to forget one.' Introduce the parameterized constructor as automatic initialization.
- Deliberately try `BankAccount` account; after only defining the parameterized constructor, and let the compiler error happen live — this is a real, common gotcha worth showing rather than just describing.
- Add a default constructor to fix it, connecting back to Module 7's overloading concept: 'multiple constructors with different parameter lists is just overloading, applied to a special kind of function.'

4. `this->` in five minutes, not more (10 min)

- Show the `Point(int x, int y)` constructor where parameter names shadow member names. Explain `this->x = x`; as 'this object's x gets the value of the parameter named x.'
- Keep this brief and practical — the only situation students need to recognize it in for now is exactly this naming collision.

5. class vs. struct — practical guidance (5–10 min)

- State the course's rule of thumb plainly: struct for simple public data bundles (no behavior needed); class when you want private data, methods, and enforced rules.

Common Student Misconceptions

Misconception	How to Address It
"Making everything private and adding a getter/setter for every field is 'doing OOP correctly.'"	Encapsulation's value is in the RULES a method can enforce (e.g. rejecting a negative deposit), not in hiding data for its own sake — a getter/setter pair with no validation logic provides little benefit over just making the field public.
"A class needs a constructor to compile."	If you write NO constructor at all, C++ generates a default one automatically — the gotcha only appears once you define ANY constructor yourself, which removes the automatic one.
"this is a variable I need to declare."	this already exists inside every non-static method automatically — it's never declared, only used.

Exercises & Module Project

Exercise: 08-classes-and-objects/exercises/exercise1.cpp — a Rectangle class with constructor, area(), perimeter(), scale(). Project: a full Bank Account Simulator with transaction history and a menu-driven main using switch (08-classes-and-objects/project/README.md).

Facilitation guidance: The Bank Account Simulator project is intentionally the most substantial single project before the capstone — it combines classes, vectors (transaction history), loops, and switch. Budget a full session (or take-home time) for it rather than treating it as a quick wrap-up exercise.

Discussion & Comprehension Check Questions

1. Why might a BankAccount class want to reject a deposit of a negative amount inside the deposit() method, rather than trusting the calling code to only pass valid amounts?

Guidance: Encapsulation lets the class itself guarantee its own rules regardless of what any caller does — trusting every caller to behave is fragile in any program larger than a single file.

2. What's the practical difference between a struct Student (Module 5) and a class Student with private fields and methods?

Guidance: The struct exposes all its data freely with no behavior; the class can hide its data and expose only validated, purposeful operations on it.

Assessment Ideas

- Design critique: give students a class with all-public fields and no methods, modeling something with an obvious invariant (e.g. a Temperature class that shouldn't go below absolute zero) and ask them to redesign it with proper encapsulation.

Answer Key Notes

See 08-classes-and-objects/exercises/solutions/exercise1_solution.cpp.

Wrap-Up & Bridge to Next Module

Bridge forward: "You now have every tool needed for the capstone — types, control flow, data structures, functions, and classes. The final module is entirely about combining them into one real, staged project, committed to GitHub the way a professional project actually grows."

Instructor Tips

- Resist introducing inheritance or polymorphism in this module even if a strong student asks — flag it explicitly as 'the natural next topic after this course' (it's named as such in the capstone's closing section) rather than a quick tangent that under-serves the core encapsulation/constructor material.